TP8: Parcours en largeur et en profondeur

CPES ≪ Sciences des données, arts et cultures ≫

Antoine GAUQUIER
antoine.gauquier@ens.fr

Pierre ABOULKER
pierreaboulker@gmail.com

16 février 2026

Nous allons dans ce TP implémenter le parcours en largeur (*Breadth First Search*) et le parcours en profondeur (*Depth First Search*), en utilisant et complétant les classes définies dans le précédent TP. Il vous faut donc dans un premier temps récupérer le code des classes de la correction du TP7 si vous ne l'avez pas encore.

Parcours en largeur en Python

Nous allons implémenter l'algorithme de *parcours en largeur*, en anglais *Breadth-First Search (BFS)*. Comme indiqué dans le cours, afin d'implémenter cet algorithme, il faut doter les *Sommet* de trois attributs supplémentaires :

— Une couleur, qui représente le statut du sommet dans le parcours : *blanc* pour un sommet qui n'a jamais été visité, *gris* pour un sommet qui a été visité dont au moins un des voisins est encore *blanc*, et *noir* pour un sommet déjà visité qui n'a pas ou plus de voisins *blancs*)

— Une distance, qui représente la distance du sommet à l'origine du parcours, à savoir, le nombre d'arêtes qu'il a fallu suivre pour atteindre ce sommet

— La parentalité du sommet, à savoir, à partir de quel sommet nous avons atteint le sommet courant.

Question 1 Nous allons d'abord créer une classe *SommetBFS*, **héritante**[1] de la classe *Sommet*. Compléter le code du constructeur de cette classe, dont le squelette du code est disponible dans `code_fourni_bfs_dfs.py`. Il vous faut ajouter trois lignes dans ce constructeur, pour initialiser les trois attributs `couleur`, `distance` et `parent`. Pour réprésenter la valeur ∞, on utilisera `math.inf` de la bibliothèque `math`.

Question 2 Créer une méthode `initialisation_bfs` pour la classe *SommetBFS*, qui initialise les attributs définis par *SommetBFS* mais pas par *Sommet*, à savoir `couleur`, `distance` et `parent`. Cette fonction sera utilisée sur tous les sommets d'un graphe avant l'exécution de l'algorithme de parcours en largeur.

1. Une classe héritante (dite classe fille) *a* d'une autre classe (dite mère) *A* intègre par héritage les attributs et méthodes de la classe *A* sans avoir à les redéfinir. En Python, on écrira `class a(A):` pour définir une telle classe.

Question 3 A partir du pseudo-code de l'algorithme de parcours en largeur fourni dans le cours à la slide 47 écrire, **en dehors de toute classe**, une fonction `parcours_en_largeur`, prenant en paramètres un objet de la classe *Graphe*, nommé `graphe`, qu'on supposera constitué de sommets *SommetBFS* (donc, on supposera que l'on aura défini le paramètre `classe_sommet = SommetBFS` lors de la construction du graphe); ainsi qu'une variable `origine`, qui représente l'`id` de l'objet de la classe *SommetBFS* à partir duquel on commence le parcours. Quelques indications pour l'écriture de cette fonction :

- Afin d'implémenter la *file* nécessaire à l'écriture de cette fonction, vous utiliserez la classe `deque` de la bibliothèque `collections`. Pour rappel, deux méthodes principales permettent de manipuler un objet `deque` en tant que *file* : la méthode `appendleft()`, qui permet d'ajouter un élément à gauche de la file, et la méthode `pop()`, qui renvoie l'élément de droite de la file **et le supprime**.
- Pensez à utiliser les méthodes déjà définies dans la classe *Graphe* afin d'implémenter la fonction. En particulier, vous aurez probablement besoin de faire appel aux méthodes `voisinage_sommet()` et `__getitem__()` (qui s'utilise avec des `[]` : par exemple pour accéder au sommet d'id `id_sommet` dans un objet de *Graphe* appelé `g`, il suffira de faire `g[id_sommet]`, et non `g.__getitem__(id_sommet)`).

Question 4 Une fois la fonction `parcours_en_largeur` implémentée, la compléter pour pouvoir observer les différentes étapes du parcours. On pourra pour cela, à chaque itération du parcours, afficher : le numéro de l'itération, l'`id` du sommet visité, la liste des sommets éventuellement ajoutés à la *file* en conséquence de la visite du sommet numéro `id`, et enfin l'état de la file à la fin de l'itération en cours. Testez ensuite votre fonction sur la graphe K_4, et sur un graphe généré aléatoirement si le temps le permets (le code permettant de générer un tel graphe est présent dans `code_fourni_bfs_dfs.py`).

Parcours en profondeur en Python

Nous allons implémenter la version *récursive* de l'algorithme de parcours en profondeur étudié en cours. Similairement que pour le parcours BFS, nous allons implémenter une nouvelle classe *SommetDFS*, héritante de la classe *Sommet*. Elle implémentera les attributs déjà utilisés dans `SommetBFS`, à savoir une couleur, une distance et une parentalité, ainsi que des attributs supplémentaires :

- un attribut *temps de découverte*, qui sera noté `d`, représentant le moment auquel le sommet a été découvert (donc grisé).
- un attribut *temps de fin*, qui sera noté `f`, représentant le moment auquel le sommet a été « fini » (donc noirci).

Question 5 Implémenter une classe *SommetDFS*, héritante de la classe *Sommet*, dans laquelle vous écrirez deux fonctions. Un constructeur, qui initialise par défaut les valeurs des attributs décrits plus haut, et une méthode `initialisation_dfs`, préparant les attributs de l'objet pour un nouveau parcours en profondeur. Vous pourrez vous inspirer de ce qui a été fait pour la classe *SommetBFS* pour écrire cette nouvelle classe qui en est très proche.

Question 6 A partir du pseudo-code de l'algorithme de parcours en profondeur *récursif* du cours, écrire :

- Une fonction `parcours_en_profondeur`, prenant en paramètres un objet *Graphe*, nommé `graphe`, qu'on supposera constitué de sommets *SommetDFS* (donc, on supposera que l'on aura défini le paramètre `classe_sommet = SommetDFS` lors de la construction du graphe). Cette fonction correspondra à la fonction DFS dans le cours.
- Une fonction `parcours_en_profondeur_sommet`, prenant en paramètres un objet *Graphe*, nommé `graphe`, qu'on supposera constitué de sommets *SommetDFS* (donc, on supposera que l'on aura défini le paramètre `classe_sommet = SommetDFS` lors de la construction du graphe), et un entier `id_sommet`, représentant l'ID du sommet à partir duquel commencer le parcours en profondeur. Cette fonction correspondra à la fonction DFS-VISITE dans le cours.

Question 7 Une fois les deux fonctions implémentées, les compléter pour pouvoir observer les différentes étapes du parcours. Il vous faudra pour cela, à chaque itération du parcours, afficher le numéro de l'itération et l'`id` du sommet visité. Testez ensuite votre fonction sur la graphe K_4 construit dans le TP7, et sur un graphe généré aléatoirement si le temps le permets (le code permettant de générer un tel graphe est présent dans le fichier `code_fourni_bfs_dfs.py` sur le Moodle du cours).